db_formula1 - script comentado

Trabalho DML (Fórmula 1) - Modelagem de Banco de Dados. Mesmo código da entrega, com explicação em cada bloco.

```sql
-- ============================================================
-- TRABALHO DML - FÓRMULA 1 (script comentado para estudo)
-- É o mesmo código da entrega, só que explicado bloco por bloco.
-- Dá para rodar inteiro no MySQL Workbench: Ctrl+Shift+Enter
-- roda tudo. Os comentários começam com "-- " e o MySQL ignora.
--
-- Deu "database exists" ou "Duplicate entry 'GAM' for key
-- 'tb_equipes.PRIMARY'"? Não é erro no código: o banco já foi
-- criado antes e o INSERT tentou cadastrar de novo uma chave que
-- já existe. Rode só esta linha uma vez:
--     DROP DATABASE db_formula1;
-- e depois rode o script inteiro de novo, do começo.
-- ============================================================


-- ------------------------------------------------------------
-- 1) CRIANDO O BANCO
-- CREATE DATABASE cria o banco vazio. O "db_" na frente é só o
-- padrão de nome que a gente usa (banco = db_, tabela = tb_).
-- ------------------------------------------------------------
CREATE DATABASE db_formula1;

-- ------------------------------------------------------------
-- 2) USANDO O BANCO
-- USE avisa o MySQL que tudo daqui pra baixo é dentro do
-- db_formula1. Sem isso ele não sabe onde criar as tabelas.
-- ------------------------------------------------------------
USE db_formula1;


-- ------------------------------------------------------------
-- 3) CRIANDO AS TABELAS
-- A ordem importa. Primeiro vem a tabela que não depende de
-- ninguém (tb_paises). Depois as que têm FOREIGN KEY apontando
-- para uma tabela que já precisa existir.
-- ------------------------------------------------------------

-- tb_paises: lista de países.
-- cod_pais é a PRIMARY KEY (chave primária): não repete e não
-- pode ficar vazio. NOT NULL obriga a preencher o nome.
CREATE TABLE tb_paises (
    cod_pais INT PRIMARY KEY,
    nome_pais VARCHAR(100) NOT NULL
);

-- tb_equipes: cada equipe tem uma sigla (PK) e o país dela.
-- FOREIGN KEY (cod_pais) REFERENCES tb_paises(cod_pais) quer
-- dizer: só aceita um cod_pais que já exista em tb_paises.
CREATE TABLE tb_equipes (
    sigla VARCHAR(10) PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cod_pais INT,
    FOREIGN KEY (cod_pais) REFERENCES tb_paises(cod_pais)
);
```

```sql

-- tb_pilotos: a PK é o número do carro (numero_piloto).
-- Tem duas FKs: o país do piloto (tb_paises) e a equipe dele
-- (tb_equipes). Por isso ela só pode ser criada depois das duas.
CREATE TABLE tb_pilotos (
    numero_piloto INT PRIMARY KEY,
    nome_piloto VARCHAR(100) NOT NULL,
    cod_pais INT,
    sigla_equipe VARCHAR(10),
    FOREIGN KEY (cod_pais) REFERENCES tb_paises(cod_pais),
    FOREIGN KEY (sigla_equipe) REFERENCES tb_equipes(sigla)
);

-- tb_corridas: cada GP tem uma sigla (PK), o nome, o país onde
-- acontece e a quantidade de voltas (INT, porque é número).
CREATE TABLE tb_corridas (
    sigla_gp VARCHAR(10) PRIMARY KEY,
    nome_gp VARCHAR(100) NOT NULL,
    cod_pais INT,
    qtd_voltas INT,
    FOREIGN KEY (cod_pais) REFERENCES tb_paises(cod_pais)
);

-- tb_participantes: liga pilotos e corridas. Um piloto corre
-- vários GPs e um GP tem vários pilotos (relacionamento N:N),
-- então precisa dessa tabela no meio.
-- A PK é composta (sigla_gp + numero_piloto): o mesmo piloto
-- não pode aparecer duas vezes no mesmo GP.
CREATE TABLE tb_participantes (
    sigla_gp VARCHAR(10),
    numero_piloto INT,
    posicao_largada INT,
    PRIMARY KEY (sigla_gp, numero_piloto),
    FOREIGN KEY (sigla_gp) REFERENCES tb_corridas(sigla_gp),
    FOREIGN KEY (numero_piloto) REFERENCES tb_pilotos(numero_piloto)
);


-- ------------------------------------------------------------
-- 4) POPULANDO AS TABELAS (INSERT) - questões 1 e 2
-- Segue a mesma ordem das FKs:
-- países -> equipes -> pilotos -> corridas -> participantes.
-- Se tentar inserir um piloto com um país que ainda não existe,
-- o MySQL dá erro de chave estrangeira.
-- Texto vai entre aspas simples ('Brasil'), número vai sem aspas.
-- ------------------------------------------------------------

-- países: código simples, 1 = Brasil, 2 = Itália, 3 = França...
INSERT INTO tb_paises (cod_pais, nome_pais) VALUES
(1, 'Brasil'),
(2, 'Itália'),
(3, 'França'),
(4, 'Estados Unidos'),
(5, 'Alemanha'),
(6, 'Mônaco');

-- equipes: a Gambiarra Racing é a equipe inventada do grupo.
-- O último valor de cada linha é o cod_pais da equipe.
INSERT INTO tb_equipes (sigla, nome, cod_pais) VALUES
('GAM', 'Gambiarra Racing', 1),
('FER', 'Ferrari', 2),
('ALP', 'Alpine', 3),
('HAS', 'Haas', 4),
('MER', 'Mercedes', 5);
```

```sql

-- QUESTÃO 1: no mínimo 5 pilotos do Brasil (cod 1), 3 da Itália
-- (cod 2) e 2 da França (cod 3). Os 2 últimos são de outros
-- países, só para a lista não ficar presa nesses três.
INSERT INTO tb_pilotos (numero_piloto, nome_piloto, cod_pais, sigla_equipe) VALUES
(7, 'Gabriel Freio de Bicicleta', 1, 'GAM'),
(13, 'Daniel Cupim de Ferro', 1, 'GAM'),
(22, 'Matheus Sem Câmbio', 1, 'GAM'),
(33, 'Pedro Verstapobre', 1, 'GAM'),
(99, 'Guilherme Drift', 1, 'GAM'),
(5, 'Alberto Ascari', 2, 'FER'),
(21, 'Giancarlo Fisichella', 2, 'FER'),
(12, 'Kimi Antonelli', 2, 'MER'),
(10, 'Pierre Gasly', 3, 'ALP'),
(31, 'Esteban Ocon', 3, 'HAS'),
(16, 'Charles Leclerc', 6, 'FER'),
(27, 'Nico Hülkenberg', 5, 'HAS');

-- corridas: sigla do GP, nome, cod_pais e número de voltas.
INSERT INTO tb_corridas (sigla_gp, nome_gp, cod_pais, qtd_voltas) VALUES
('BRA', 'GP do Brasil', 1, 71),
('ITA', 'GP da Itália', 2, 53),
('FRA', 'GP da França', 3, 53),
('EUA', 'GP dos Estados Unidos', 4, 56),
('MON', 'GP de Mônaco', 6, 78);

-- participantes: quem largou em qual GP e em que posição.
-- Só dá para inserir aqui depois que o piloto e a corrida
-- já existem nas tabelas deles.
INSERT INTO tb_participantes (sigla_gp, numero_piloto, posicao_largada) VALUES
('BRA', 33, 1),
('BRA', 10, 2),
('BRA', 12, 3),
('BRA', 31, 4),
('BRA', 21, 5),
('BRA', 5, 6),
('BRA', 99, 7),
('BRA', 22, 8),
('BRA', 13, 9),
('BRA', 7, 10),
('ITA', 5, 1),
('ITA', 21, 2),
('ITA', 12, 3),
('ITA', 10, 4),
('ITA', 33, 5),
('ITA', 27, 6),
('MON', 10, 1),
('MON', 31, 2),
('MON', 12, 3),
('MON', 99, 4),
('MON', 7, 5),
('MON', 16, 6);


-- ------------------------------------------------------------
-- 5) CONSULTAS (SELECT)
-- Estrutura básica: SELECT colunas FROM tabela WHERE condição
-- ------------------------------------------------------------

-- QUESTÃO 3: países que começam com B e terminam com L.
-- No LIKE, o % vale "qualquer coisa no meio". 'B%L' = começa
-- com B, termina com L. O MySQL não diferencia maiúscula de
-- minúscula aqui, então 'Brasil' entra.
SELECT nome_pais FROM tb_paises WHERE nome_pais LIKE 'B%L';
```

```sql

-- QUESTÃO 4: pilotos do Brasil, Itália e França.
-- IN (1, 2, 3) é o mesmo que cod_pais = 1 OR cod_pais = 2
-- OR cod_pais = 3, só que mais curto.
SELECT nome_piloto FROM tb_pilotos WHERE cod_pais IN (1, 2, 3);

-- QUESTÃO 5: só o nome de todos os pilotos (sem WHERE = todos).
SELECT nome_piloto FROM tb_pilotos;

-- QUESTÃO 6: ALIAS (apelido de coluna).
-- AS troca o nome que aparece no cabeçalho do resultado.
-- "Grande Prêmio" tem espaço, por isso vai entre aspas duplas.
-- voltas é uma palavra só, então vai sem aspas.
SELECT nome_gp AS "Grande Prêmio", qtd_voltas AS voltas FROM tb_corridas;

-- QUESTÃO 7: nome do piloto e nome do país (duas tabelas).
-- As duas tabelas vão no FROM e o WHERE liga pelo campo que
-- elas têm em comum (cod_pais). Escreve tabela.coluna porque
-- cod_pais existe nas duas. Sem esse WHERE, cada piloto sairia
-- combinado com todos os países.
SELECT tb_pilotos.nome_piloto, tb_paises.nome_pais
FROM tb_pilotos, tb_paises
WHERE tb_pilotos.cod_pais = tb_paises.cod_pais;
-- Com INNER JOIN dá o mesmo resultado:
--   SELECT tb_pilotos.nome_piloto, tb_paises.nome_pais
--   FROM tb_pilotos INNER JOIN tb_paises
--   ON tb_pilotos.cod_pais = tb_paises.cod_pais


-- ------------------------------------------------------------
-- 6) MUDANDO A TABELA E OS DADOS
-- ------------------------------------------------------------

-- QUESTÃO 8: criar a coluna salario numa tabela que já existe.
-- ALTER TABLE ... ADD acrescenta a coluna nova.
-- DECIMAL(10,2) = até 10 dígitos, 2 depois da vírgula. É o
-- tipo certo para dinheiro (FLOAT pode arredondar errado).
ALTER TABLE tb_pilotos ADD salario DECIMAL(10,2);

-- QUESTÃO 9: a coluna nova nasce vazia (NULL) para todo mundo.
-- UPDATE ... SET muda o valor e o WHERE escolhe qual piloto.
-- Cuidado: UPDATE sem WHERE muda TODAS as linhas da tabela.
-- O 1500.00 se repete de propósito (pilotos 7 e 13), para a
-- questão 12 mostrar o DISTINCT funcionando.
UPDATE tb_pilotos SET salario = 1500.00 WHERE numero_piloto = 7;
UPDATE tb_pilotos SET salario = 1500.00 WHERE numero_piloto = 13;
UPDATE tb_pilotos SET salario = 50000.00 WHERE numero_piloto = 22;
UPDATE tb_pilotos SET salario = 100000.00 WHERE numero_piloto = 33;
UPDATE tb_pilotos SET salario = 75000.00 WHERE numero_piloto = 99;
UPDATE tb_pilotos SET salario = 120000.00 WHERE numero_piloto = 5;
UPDATE tb_pilotos SET salario = 80000.00 WHERE numero_piloto = 21;
UPDATE tb_pilotos SET salario = 70000.00 WHERE numero_piloto = 12;
UPDATE tb_pilotos SET salario = 95000.00 WHERE numero_piloto = 10;
UPDATE tb_pilotos SET salario = 90000.00 WHERE numero_piloto = 31;
UPDATE tb_pilotos SET salario = 110000.00 WHERE numero_piloto = 16;
UPDATE tb_pilotos SET salario = 85000.00 WHERE numero_piloto = 27;
```

```sql


-- ------------------------------------------------------------
-- 7) CONSULTAS COM SALÁRIO
-- ------------------------------------------------------------

-- QUESTÃO 10: salários entre 70 e 100 mil.
-- BETWEEN 70000 AND 100000 inclui os dois limites, então quem
-- ganha exatamente 70000 ou 100000 também aparece.
-- Número no BETWEEN vai sem ponto de milhar e sem R$.
SELECT nome_piloto, salario FROM tb_pilotos WHERE salario BETWEEN 70000 AND 100000;

-- QUESTÃO 11: média salarial para cada cod_pais.
-- AVG calcula a média. GROUP BY cod_pais junta os pilotos do
-- mesmo país e faz uma média para cada grupo. O enunciado fala
-- "empregados", mas aqui são os pilotos.
-- AS media_salarial só dá nome para a coluna calculada.
SELECT cod_pais, AVG(salario) AS media_salarial FROM tb_pilotos GROUP BY cod_pais;

-- QUESTÃO 12: valores diferentes de salário, do menor pro maior.
-- DISTINCT tira as repetições (o 1500.00 aparece uma vez só).
-- ORDER BY salario ASC ordena crescente (ASC já é o padrão,
-- mas deixei escrito porque a questão pede ordem crescente).
SELECT DISTINCT salario FROM tb_pilotos ORDER BY salario ASC;
```